

Structural Convergence and Parameter-Efficient Continual Adaptation: Theoretical Mechanisms and Reinforcement Learning Applications

Introduction to Lifelong Learning and Structural Parameterization

The trajectory of large-scale artificial intelligence has increasingly shifted from static, isolated model training toward dynamic, lifelong learning paradigms. As foundation models scale to hundreds of billions of parameters, the computational, energetic, and financial costs associated with fully retraining models for novel tasks have become fundamentally prohibitive. This limitation has necessitated the rapid development of parameter-efficient fine-tuning (PEFT) techniques, which seek to adapt large neural networks to new distributions without modifying the bulk of the pretrained weights. However, the deployment of continuous adaptation mechanisms in real-world, sequential settings faces two interrelated theoretical and logistical bottlenecks. First, the proliferation of independent, task-specific adapters scales linearly with the number of tasks encountered, creating severe memory constraints and management overhead in large-scale multi-tenant serving environments. Second, and more fundamentally, sequential adaptation is continuously plagued by catastrophic forgetting—a phenomenon where the integration of novel parametric information systematically overwrites the topological representations of previously acquired capabilities.

Recent empirical and theoretical advancements suggest that the ultimate solution to these lifelong learning bottlenecks resides in the inherent geometric properties of deep neural networks. Rigorous spectral analyses indicate that neural architectures, regardless of their random initialization or the specific domain data they are trained upon, systematically converge toward shared, low-dimensional parametric subspaces. This geometric universality implies that fine-tuning is not an unconstrained traversal across boundless high-dimensional parameter spaces, but rather a localized, highly constrained optimization within an intrinsically restricted manifold. By formalizing this convergence, novel methodologies such as Parameter-Efficient Continual Finetuning (PaCT) have emerged. These frameworks leverage shared low-rank subspaces to enable near-strict continual learning without the need for cumbersome data replay buffers, dynamically updating a singular parametric basis while preserving historical task coefficients.

The implications of these structural discoveries extend far beyond static natural language processing or isolated computer vision tasks. In the domain of reinforcement learning (RL),

where agents must navigate highly non-stationary data distributions governed by their own continuously evolving behavioral policies, catastrophic forgetting is particularly acute. The application of low-rank adaptation to proximal policy optimization (PPO) and complex actor-critic architectures offers a mechanism to stabilize value functions and policy updates while drastically reducing post-training computational overhead. When applied to highly complex, procedurally generated environments such as the NetHack Learning Environment (NLE) and MiniHack, these parameter-efficient continual learning frameworks provide a theoretical and practical foundation for solving the exploration-exploitation dilemma across an infinite array of task variations.

This report conducts an exhaustive theoretical and empirical analysis of the Universal Weight Subspace Hypothesis and the PaCT framework, specifically examining the Share (Shared LoRA Subspaces) methodology. It subsequently bridges these mathematical formulations with the domain of reinforcement learning, explicitly analyzing the forgetting mitigation problem in sequential decision-making. Finally, the analysis explores the architectural extension of these methods to the recurrent, convolutional baselines required to master the procedural complexities of the NetHack Learning Environment, outlining a comprehensive blueprint for lifelong learning in deep reinforcement learning systems.

Fundamentals of Low-Rank Adaptation (LoRA) and Reinforcement Learning Integration

To thoroughly analyze the PaCT framework and its applications, it is necessary to first establish the mathematical foundation of standard Low-Rank Adaptation (LoRA) and how it interacts with the optimization dynamics of Reinforcement Learning (RL). LoRA was engineered to democratize the fine-tuning of massive foundation models by hypothesizing that the changes in weights during task adaptation possess a low intrinsic rank.¹

The Mathematics of Standard LoRA

Given a pretrained weight matrix $W_0 \in \mathbb{R}^{n \times d}$, full fine-tuning computes a dense gradient matrix $\Delta W \in \mathbb{R}^{n \times d}$ such that the updated weights become $W = W_0 + \Delta W$. For highly overparameterized models, optimizing and storing ΔW for every downstream task is entirely inefficient. LoRA addresses this by constraining the update matrix ΔW through a low-rank decomposition. Specifically, it introduces two trainable matrices $B \in \mathbb{R}^{n \times r}$ and $A \in \mathbb{R}^{r \times d}$, where the rank $r \ll \min(n, d)$.³

The forward pass for a modified layer under the LoRA framework is mathematically expressed as:

$$h = W_0x + \Delta Wx = W_0x + \frac{\alpha}{r}BAx$$

Here, α is a constant scaling factor, and r is the specified rank. During optimization, the original weights W_0 remain frozen and receive no gradient updates, while only A and B are optimized.² Matrix A is typically initialized with random Gaussian noise, while matrix B is initialized with zeros, ensuring that at the onset of training, $\Delta W = 0$ and the model's behavior is identical to the base pretrained model.³ This framework reduces the number of trainable parameters by orders of magnitude, mitigating overfitting and minimizing the data demands required to reach convergence.¹

Integrating LoRA with Policy Gradient Methods

While LoRA is a standard procedure for Supervised Fine-Tuning (SFT), its application to Reinforcement Learning introduces unique theoretical considerations. RL optimization is characterized by high variance and non-stationary target distributions, making the stability of the function approximator paramount. When utilized within Proximal Policy Optimization (PPO), LoRA restricts the actor and critic network updates strictly to the low-rank subspace.⁴ Empirical investigations into RLVR (Reinforcement Learning with Verifiable Rewards) demonstrate that LoRA is exceptionally well-suited for policy gradient methods. The efficacy of LoRA in RL is fundamentally governed by the intrinsic density of the learning signal.⁶ SFT often attempts to inject massive amounts of fundamentally novel, "high-rank" factual knowledge into a model (e.g., memorizing specific biological pathways absent from the pretraining corpus), which can saturate the capacity of low-rank matrices.⁶ In contrast, RLVR and PPO typically operate on sparse outcome rewards derived from experiential trajectories.⁶ Because the RL objective is fundamentally aligning existing latent representations toward a specific behavioral heuristic—rather than absorbing dense factual data—the capacity requirements are uniquely low. As a result, LoRA fully matches the learning performance of Full Fine-Tuning (FullFT) when executing policy gradients, while requiring vastly fewer computational resources.⁶ The integration of LoRA into distributed RL frameworks is supported by advanced multi-tenant serving infrastructures. Libraries such as verl allow for the parallelized deployment of PPO with LoRA across FSDP (Fully Sharded Data Parallel) and Megatron backends, enabling the fine-tuning of very large models (e.g., 70B+ parameters) on modest hardware configurations.⁵ This architecture simplifies model transfer and deployment, as only the lightweight LoRA adapters need to be checkpointed and loaded during rollout generation and optimization phases.⁵

Empirical Deployments of LoRA in RL

Recent literature explicitly demonstrates the viability of applying LoRA to complex RL scenarios, particularly in multi-agent reasoning and logic environments.

Study / Model	Domain / Task Focus	Key Empirical Findings and Performance Metrics
Budget-Aware Allocation (ToolBench)	Multi-agent Sequential Decision Problem	Hybrid policy combining heuristic traces with PPO-tuned LoRA subspaces. Achieved perfect success while reducing Cost-Per-Success (CPS) by 12% relative to full PPO (21.40 vs. 24.30). ⁴
FlightPlanner Environment	Synthetic Reasoning	Achieved the lowest CPS (7.71) among high-success methods, significantly outperforming rule-based baselines (10.58) and supervised LoRA (11.70). ⁴
Tina Models (1.5B Base)	Mathematical Reasoning (AIME24)	Achieved a 43.33% Pass@1 accuracy on AIME24 with a tiny 1.5B parameter base model, costing only \$9 USD for post-training and evaluation, representing a 260x cost reduction compared to SOTA RL reasoning models. ⁸

These results suggest that restricting updates to the adapter stabilizes optimization, radically improves sample efficiency, and preserves structural heuristics while enabling sequential credit assignment.⁴ The success of the Tina models, in particular, confirms the hypothesis that the effectiveness of efficient RL stems from LoRA rapidly adapting the model to the structural format of reasoning rewarded by RL, while completely preserving the base model's underlying knowledge within the frozen high-dimensional weights.⁸

The Universal Weight Subspace Hypothesis

To understand how continual fine-tuning can be theoretically guaranteed to avoid interference across thousands of tasks, it is necessary to examine the geometric properties of the parameter

space itself. The Universal Weight Subspace Hypothesis provides this exact theoretical lens, positing a fundamental geometric constraint on the optimization trajectories of deep neural networks.⁹ The hypothesis asserts that deep neural networks trained across diverse, potentially disjoint, and unrelated tasks exhibit remarkably similar low-dimensional parametric subspaces.⁹

Scale of Empirical Validation and Spectral Decay

Through large-scale mode-wise spectral analysis encompassing over 1,100 distinct neural models, empirical evidence demonstrates that networks systematically converge to shared spectral subspaces, regardless of initialization, task formulation, domain, or optimization hyperparameters.¹²

The scale of this empirical validation is unprecedented. The analysis spans across multiple architectures and modalities, confirming that geometric universality is not an artifact of a specific model family, but a fundamental property of deep learning optimization:

- **Vision Transformers (ViT):** 500 ViT models with completely random initializations, trained on entirely disjoint image datasets with diverse hyperparameter settings, were analyzed in full weight space.⁹
- **Mistral-7B LoRAs:** 500 independent LoRA adapters trained on highly diverse NLP instruction-following and generation tasks.⁹
- **LLaMA-8B / LLaMA3-8B:** 50 foundation models analyzed in full weight space across complex text understanding metrics.¹¹
- **Generative Models and Traditional LMs:** Principal component analysis of 200 GPT-2 models, 8 Flan-T5 models, and Stable Diffusion-XL LoRAs consistently revealed sharp spectral decay.⁹

The spectral data indicates that when decomposition is applied to the weight matrices of these models, the vast majority of the spectral variance (often 90-95%) is captured by a highly restricted number of principal components, frequently ranging from just 16 to 32 dimensions, despite the underlying parameter matrices possessing dimensions in the thousands.¹⁰ If the representation geometry were determined primarily by the underlying data, models trained on medical imaging would exhibit orthogonal spectral directions compared to those trained on satellite photography or natural language processing tasks. However, the empirical results show the exact opposite: the networks collapse into the exact same parametric subspace, proving that the network's architecture and the optimization algorithm dictate its geometry far more significantly than the training data.⁹

Mathematical Formulation of Universal Convergence

To formalize this hypothesis, predictors are mathematically modeled as elements of a separable Hilbert space $(H, \langle \cdot, \cdot \rangle)$ with a defined norm $\| \cdot \| = \| \cdot \|_H$.⁹ This framework allows for the rigorous analysis of generalization and inductive biases. For any elements $a, b \in H$, the rank-one operator $a \otimes b : H \rightarrow H$ is defined as $(a \otimes b)g = \langle b, g \rangle a$, where the

operator norm is bounded by $\|a \otimes b\|_{op} = \|a\| \|b\|$.⁹

Assuming a scenario where T tasks are drawn independently and identically distributed from a task distribution $\mathcal{T}$, each task t possesses an unknown ground-truth predictor $f_t^* \in H$ and a learned predictor $\hat{f}_t \in H$.⁹ The convergence to a universal subspace is analyzed through the lens of task second-moment operators. The population task second-moment operator is defined as:

$$S := E_{t \sim \mathcal{T}}[f_t^* \otimes f_t^*]$$

Concurrently, the true empirical task second-moment operator $\hat{S}$ and the learned empirical task second-moment operator $\tilde{S}$ are given by:

$$\hat{S} := \frac{1}{T} \sum_{t=1}^T f_t^* \otimes f_t^*, \quad \tilde{S} := \frac{1}{T} \sum_{t=1}^T \hat{f}_t \otimes \hat{f}_t$$

These operators S , $\hat{S}$, and $\tilde{S}$ are self-adjoint and positive semi-definite with a finite trace $\text{tr}(S) < \infty$.⁹ The top- k eigenspace of these operators represents the population rank- k shared subspace. The Theorem of Two-level Convergence to the Shared Subspace dictates that the empirical second-moment operator built from learned predictors reliably converges to the true operator S , and the learned top- k subspace $\hat{P}_k$ converges to the true subspace P_k .¹⁶

The rate of this convergence mathematically bounds two distinct sources of error: the error associated with averaging across T tasks (which scales proportionally to $1/T$), and per-task estimation noise parameters ($\bar{\eta}$ and $\bar{\eta}^2$).¹⁶ For absolute constants c_1, c_2 and a given probability limit $\delta \in (0, 1)$, the bounds on the operator norm $\|\tilde{S} - S\|_{op}$ mathematically prove that as task diversity increases, the spectral boundary of the hypothesis space rigidifies.¹¹ In applied experimental contexts, researchers extract the eigenvectors of the empirical operator $\tilde{S}$ utilizing Truncated Zero-Centered Higher-Order Singular Value Decomposition (HOSVD).

This algorithm operates on a high-order tensor $\mathcal{X} \in \mathbb{R}^{I_1 \times \dots \times I_N}$ constructed by stacking the

individual rank-reduced task matrices along mode n , efficiently isolating the universal components.¹⁶

The Three Forces of Geometric Universality

The universality of these low-rank geometric structures is not a stochastic artifact, but is instead driven by three interacting computational forces inherent to deep learning:

1. **Spectral Bias:** Gradient-based optimization algorithms (such as Stochastic Gradient Descent or Adam) possess an inherent spectral bias, preferentially fitting low-frequency, low-rank modes in early training phases. This bias channels diverse learning trajectories toward smooth solutions and shared geometric manifolds, leading to rapid spectral decay in the weight matrices.¹⁰
2. **Architectural Inductive Bias:** The physical topology of the network imposes rigid structural constraints. For example, convolutional structures strictly favor specific localized filter types (e.g., Gabor-like filters consistently learned in the primary layers of CNNs), while self-attention layers in Transformers naturally induce low-rank bottleneck structures in token-to-token interactions.⁹
3. **The NTK Regime:** In the infinite-width limit of neural networks, the Neural Tangent Kernel (NTK) governing the training dynamics becomes fundamentally task-independent.⁹ Because the kernel becomes fixed and invariant to task specifics, the optimization dynamics across completely disparate training tasks share an identical underlying functional derivative space.¹⁸

Implications for Reusability and Model Merging

The identification of universal weight subspaces carries profound analytical deductions for computational efficiency. If the weights of a neural network exist almost entirely within a shared, low-dimensional subspace capturing 95% of spectral variance, storing the full high-dimensional weight matrices for multiple tasks is mathematically redundant.¹⁰ By representing each unseen model as sparse coefficients projected onto the universal basis, engineers can achieve up to 100x compression with minimal degradation in performance, vastly reducing the carbon footprint and memory overhead of large-scale models.⁹

Furthermore, this structural alignment theoretically optimizes model merging. Traditional model merging techniques—such as Task Arithmetic or TIES—often struggle with parameter interference because they attempt to merge weights in the native high-dimensional space. Because independent initializations appear orthogonal in high-dimensional space, naive averaging destroys capability. However, by leveraging the universal subspace, models can be seamlessly merged without access to the original training data.⁹ The process involves strictly aligning the spectral directions within the shared subspace, effectively side-stepping the destructive interference that plagues non-orthogonal high-dimensional weight averaging.⁹

Parameter-Efficient Continual Finetuning (PaCT): The Share Framework

While the Universal Weight Subspace Hypothesis provides the theoretical justification for why neural networks are highly compressible, realizing these efficiencies in active, sequential learning environments requires explicit algorithmic execution. Adapting large pretrained models continually is notoriously challenged by the high cost of retraining and the ubiquitous threat of catastrophic forgetting.¹⁹

As previously noted, when deployed in continual learning scenarios, standard LoRA requires the instantiation of an independent adapter for every novel task. This independent scaling creates a

linear $O(T)$ parameter and memory burden, rendering it unsustainable for asynchronous learning across thousands of tasks.³ Furthermore, isolated LoRA adapters lack mechanisms for cross-task knowledge integration, preventing positive transfer between related domains.²² To definitively resolve this, the Parameter-Efficient Continual Finetuning (PaCT) paradigm has been operationalized through the *Share* framework (Shared LoRA Subspaces for almost Strict Continual Learning).³ *Share* dynamically learns and continuously updates a single, shared low-rank subspace, replacing hundreds of task-specific adapters with one evolving, foundational manifold that mitigates interference analytically.³

The Three-Phase Algorithmic Mechanics of Share

The *Share* methodology operates without relying on data replay, completely circumventing the massive storage and privacy constraints typically associated with continual learning buffers.¹⁹

The algorithmic flow is governed by three rigorous, mathematically defined phases:

Phase 1: Initialization of the Foundational Subspace The algorithm begins by constructing a shared low-rank basis that captures the core knowledge of the pretrained system or an initial

batch of tasks. *Share* initializes the foundational subspace using $t \geq 1$ LoRA adapters, where

a larger initial pool t yields a more representative foundational subspace that more closely approximates the true universal subspace boundary.³ Using Singular Value Decomposition (SVD), the principal basis vectors are extracted from these initial adapters. This process acts as a mathematical sieve, isolating the essential "axes" that the adapters utilize to adjust the frozen model, discarding lower-level singular values that largely represent dataset-specific noise.¹⁹

Phase 2: Continual Adaptation As novel tasks arrive sequentially, *Share* does not spawn

entirely new A and B matrices. Instead, it detects the specific, essential directions required for the new task by projecting the new data requirements into the existing subspace.¹⁹ During this phase, only the task-specific coefficients associated with a small subset of the established subspace directions—or occasionally newly added temporary basis vectors if the task fundamentally expands the required rank—are updated via backpropagation.¹⁹ Crucially, the principal basis vectors remain strictly frozen during the immediate fine-tuning step. This architectural freeze prevents catastrophic interference with the geometric alignment of prior tasks, ensuring that the foundational manifold is not distorted by transient optimization gradients.²⁴

Phase 3: Merging, Updating, and Closed-Form Reprojection The final phase is the most

mathematically critical for achieving "almost strict continual learning." Once the new task directions and coefficients are learned, they must be permanently merged into the persistent shared subspace. Share utilizes analytical SVD updates to merge the new directions.¹⁹ To ensure that no prior knowledge is corrupted by this basis expansion, the algorithm executes an analytical reprojection. All previously learned task coefficients for historical tasks are recalculated via closed-form projection using the Moore-Penrose pseudo-inverse.³ By applying the Moore-Penrose pseudo-inverse of the updated basis matrices (e.g., calculating β^{t+1}), the algorithm computes the exact task coefficients ϵ_{β}^{t+1} that minimize the algebraic reconstruction error between the historical adaptation state and the new, expanded subspace.³ Because this is an analytical, deterministic matrix operation rather than a stochastic gradient update subject to optimization drift, catastrophic interference is mathematically neutralized within the bounds of the subspace rank.

Ablations and Empirical Efficiency of the PaCT Framework

The architectural efficiency yielded by Share represents a paradigm shift in lifecycle model management. The configuration of the algorithm relies on specific hyperparameters: the number of Share basis vectors (k), a projection scalar (p), and thresholding variables (φ) to govern when temporary basis vectors are permanently integrated.³

Performance Metric	Share / PaCT Results vs Standard LoRA Baselines	Underlying Mechanistic Cause
Parameter Reduction	Up to 100x reduction ³	Storing only scalar coefficients and a single basis rather than full $n \times$ and $r \times$ matrices per task.
Memory Savings	Up to 281x savings ³	Elimination of redundant optimizer states and gradient buffers across isolated adapters in multi-tenant systems.
Catastrophic Forgetting	Near-zero degradation on historical tasks ²⁰	Analytical reprojection using Moore-Penrose pseudo-inverses minimizes exact historical

		reconstruction error. ³
Knowledge Transfer	Active Forward and Backward Transfer observed ³	Projection of novel tasks into the shared subspace naturally leverages previously learned principal directions, aligning cross-task heuristics. ³

The Share framework has been rigorously evaluated across a massive spectrum of domains, including natural language understanding (e.g., the GLUE continual learning benchmark), image classification, 3D object pose estimation, mathematical reasoning, and text-to-image generative diffusion models (e.g., SDXL).³ In text-to-image generation specifically, applying the universal subspace reprojection preserves visual quality and style across sequential adaptations, occasionally even outperforming individual LoRAs due to the denoising effects of spectral truncation.¹¹ In all instances, a single Share model successfully replaced hundreds of task-specific adapters, maintaining aggregate performance strictly comparable to highly expensive, jointly trained models.³

Catastrophic Forgetting in RL and Multitask Environments

While PaCT and Universal Subspaces demonstrate profound stability in supervised and generative domains, their application to Reinforcement Learning (RL) introduces significant algorithmic causality constraints. Unlike supervised learning, where the data distribution is static and externally curated, an RL agent's data distribution is heavily non-stationary, directly dictated by the ongoing permutations of its own policy. Consequently, stabilizing RL optimization during sequential task adaptation is exceptionally difficult.

Despite the inherent protection offered by freezing base parameters in standard PEFT, the trainable adapters themselves remain highly susceptible to catastrophic interference over continuous RL trajectories. Quantitative scaling laws derived from PEFT architectures reveal a mathematically strict inverse linear relationship between fine-tuning performance and the amount of catastrophic forgetting.²⁹ Forgetting in LoRA adapters increases as a shifted power law in relation to the number of parameters fine-tuned and the number of update steps.²⁹ If left unchecked in a multitask RL environment, standard LoRA will rapidly degrade.

The Forgetting Mitigation Problem in RL

Extensive research by Wolczyk et al. fundamentally reframes the challenge of multitask RL: fine-tuning reinforcement learning models is secretly a forgetting mitigation problem.³⁰ When a pretrained agent is deployed into a downstream task, standard fine-tuning frequently fails to outperform training from scratch, severely undermining the value of foundation models in RL.³⁰ This failure is not due to a lack of capacity, but because the exposure to novel states rapidly overwrites the very pretrained capabilities the fine-tuning sought to leverage. This phenomenon,

termed Forgetting of Pre-trained Capabilities (FPC), manifests predominantly in two distinct analytical forms:

1. **The State Coverage Gap:** This occurs when the state space of an RL Markov Decision Process (MDP) contains distinctly clustered regions, such as "Close" and "Far" states.³² Consider a robotic agent pretrained to manipulate distant objects (Far). In a fine-tuning task where the object is hidden inside a proximal drawer (Close), the agent initially lacks the behavioral policy to open the drawer. As it continuously explores the "Close" state to learn the drawer mechanics, it is entirely starved of exposure to the "Far" state. During this exposure drought, the network catastrophically forgets its optimal manipulation policy for the "Far" state due to interference in the function approximator.³⁰ Once the drawer is finally opened, the agent can no longer manipulate the object it originally knew how to handle.
2. **The Imperfect Cloning Gap:** In this scenario, the pretrained agent is competent in both regions, but subtle variations in the downstream environment dynamics skew the visitation distribution. This causes a disproportionate clustering of updates in one region that silently degrades the policy for the unvisited region, despite the agent theoretically possessing the knowledge to traverse it.³³

Traditional Countermeasures vs. PaCT

To counter these gaps, RL researchers actively deploy knowledge retention techniques traditionally reserved for continual learning literature. Common methodologies include:

- **Elastic Weight Consolidation (EWC):** Applying penalty terms to changes in parameters deemed critical to previous tasks.³¹
- **Kickstarting (KS):** A technique similar to behavioral cloning, where data is sampled from the current policy during fine-tuning. It computes an auxiliary loss relying on the KL divergence between the current policy and the pretrained policy: $\mathcal{L}_{KS}(\theta) = \mathbb{E}_{s \sim \pi_\theta}$, effectively anchoring the new policy's distribution to prevent the unlearning of unvisited states.³³
- **High-Entropy Token Selection:** Dynamically selecting high-entropy examples for SFT loss calculation to maintain distribution breadth.³⁴

While Wolczyk et al. demonstrated that employing these retention methods doubles state-of-the-art performance in complex environments³⁰, these techniques require computationally expensive auxiliary loss calculations and continuous KL divergence monitoring. The Share / PaCT framework offers a far more elegant structural solution to the State Coverage Gap. Because Share operates by strictly freezing the principal basis during adaptation (Phase 2) and recalculating coefficients via analytical reprojection (Phase 3), catastrophic forgetting is mitigated purely through algebraic geometry.³ If the agent is in the "Close" state learning to open the drawer, the gradient updates strictly alter temporary basis vectors or isolated coefficients. When merged, the Moore-Penrose pseudo-inverse guarantees that the optimal coefficients for the "Far" state (object manipulation) are geometrically preserved relative to the expanded subspace.¹⁹ Thus, PaCT automates knowledge retention structurally, shifting the burden from

auxiliary KL penalty loss functions directly into the immutable mathematics of subspace reprojection.

Extension and Testing in the NetHack Learning Environment (NLE) and MiniHack

The ultimate crucible for testing parameter-efficient continual learning and forgetting mitigation resides in environments characterized by infinite procedural variation, deep stochasticity, and punishingly complex state spaces. The NetHack Learning Environment (NLE) and its derivative sandbox framework, MiniHack, represent the pinnacle of such testing grounds.³⁶

The Procedural Complexity of NLE and MiniHack

NetHack is a fundamentally chaotic, procedurally generated dungeon-crawler. It features complex navigation, intricate tool utilization, deep item identification mechanics, and extreme long-term credit assignment challenges.³⁸ The stochastic nature of the game guarantees that no two episodes are identical.

MiniHack leverages the core NetHack engine to provide a domain-specific language (DSL) based on probabilistic programming.³⁷ This DSL permits researchers to generate highly diverse gymnasium environments using minimal code, controlling entity placement, terrain geometry, trap locations, and custom reward functions.³⁷ Crucially, the DSL has first-class support for underspecifying environment variables. When a researcher underspecifies a room layout, the engine injects severe procedural randomness at every reset, continuously bombarding the RL agent with previously unseen topological situations.³⁹

This procedural content generation actively induces the state coverage gap identified by Wolczyk et al. An agent might spend hundreds of episodes learning to fight monsters in narrow corridors, entirely forgetting open-room evasion tactics, before the random seed suddenly shifts the environment back to open-room geometries.

Baseline Architectures: The Chaotic Dwarven GPT-5

Successfully navigating NLE requires complex, multimodal neural architectures capable of parsing highly varied observation streams. The standard baseline for high-throughput model-free policy gradient methods in NLE is derived from the "Chaotic Dwarven GPT-5" architecture, frequently implemented via TorchBeast (IMPALA) or Sample Factory (which scales up to 33M parameters).³²

This architecture must process multiple concurrent modalities: symbolic text, discrete character matrices representing the map (glyphs), bottom-line statistics (blstats, such as hit points, experience, and hunger), and message histories.⁴¹ The core processing pipeline involves:

1. **Convolutional Encoders:** The factorized glyphs representing the top-down visual map are passed into a highly structured Convolutional Neural Network (CNN). The CNN typically features 3 distinct convolutional layers (e.g., 32, 64, and 128 feature maps) interleaved with Exponential Linear Unit (ELU) activations.⁴⁰
2. **Dense Embeddings:** Concurrently, fully connected dense networks process the

bottom-line statistics and the character-CNN encoded game messages.⁴⁰

3. **Recurrent Temporal Aggregation:** The concatenated features from the CNN and dense networks are fed into a large, hierarchical Long Short-Term Memory (LSTM) network—often consisting of 2 layers with a massive hidden dimension of 2048 and strict dropout parameters.⁴³ The LSTM aggregates the temporal sequence and outputs to separated policy (actor) and baseline (critic) value heads.³²

The presence of the CNN and LSTM in the Chaotic Dwarven architecture makes it theoretically ideal for testing the Universal Weight Subspace Hypothesis and PaCT. Prior spectral studies confirm that early convolutional layers rigidly learn structured frequency filters across widely varied tasks, while LSTM gating mechanisms enforce dense but highly correlated relational structures, both heavily relying on low-rank topologies.⁹

Extending PaCT to Mitigate RL Forgetting in NLE

Integrating the PaCT / Share framework into the Chaotic Dwarven architecture provides a theoretically robust mechanism for conquering the NLE state coverage gap. The methodology for extending this framework involves explicit modifications to the training loop and architecture. When an agent is trained across a diverse suite of MiniHack tasks—ranging from navigating complex mazes to learning the syntax of in-game menu systems to consume a specific gemstone (e.g., the "pick me up" task)³⁶—the standard LSTM and CNN weights suffer from representation drift. The proposed extension involves freezing the core Chaotic Dwarven weights entirely. The Share framework's evolving LoRA subspace is then injected directly into the query, key, value, and projection matrices of the LSTM, as well as the dense projection heads of the CNN.

1. Zero-Shot Procedural Adaptation via Subspace Geometry

If the Universal Weight Subspace Hypothesis holds for RL actors, then the diverse procedural tasks in MiniHack do not require fundamentally distinct policy representations; they simply require the activation of different coordinates within the same low-rank geometry.⁹ Instead of the PPO algorithm violently shifting the high-dimensional weights of the LSTM to account for a new randomized dungeon layout, the Shared LoRA subspace restricts the optimization trajectory. The actor-critic updates are analytically forced to traverse pre-established spectral directions.¹⁹ Because the principal basis vectors of the Share subspace capture the core structural invariants of the NetHack mechanics (e.g., collision physics, basic combat pacing, inventory management), the sample efficiency of the PPO algorithm will dramatically accelerate on unseen seeds.⁴

2. Nullifying the State Coverage Gap in NLE

The most critical advantage of extending PaCT into NLE is the application of Share's Phase 3 merging algorithm to counter the catastrophic forgetting associated with procedural shifts. Assume an NLE agent achieves high competence in navigating lava domains (State A), and then the MiniHack generator initiates an extended sequence of water-navigation tasks (State B). In standard FullFT or independent LoRA RL, the prolonged absence of lava environments

causes the value network to forget the negative reward associated with lava, and the policy network to unlearn the precise keystrokes required to avoid it.

By utilizing Share, as the agent learns water navigation (State B), the algorithm identifies the new essential directions and learns the coefficients specifically for water traversal.¹⁹ When these new vectors are integrated into the principal subspace, Share deploys the Moore-Penrose pseudo-inverse to analytically recalculate the coefficients for all prior tasks (including State A, lava navigation).³ Because this reprojection minimizes the algebraic reconstruction error of the old adapter relative to the expanded basis, the exact parameters governing lava navigation are mathematically preserved. The agent incurs virtually zero catastrophic forgetting, regardless of how long the procedural generator deprives it of the State A experience.²⁰

Evaluation Metrics and Benchmarking

To properly test this extension, performance must be rigorously benchmarked against established baselines (such as E3B checkpoints or standard Behavioral Cloning) across specific NLE metrics.³⁸ The evaluation framework must track two primary dimensions:

1. **In-Game Score:** The traditional proxy metric for optimization in Offline RL agents.⁴⁴
2. **Death Level:** A metric that lower-bounds early-game progress and is highly indicative of game completion capability.⁴⁴

Furthermore, performance should be evaluated using normalized scores against the state-of-the-art symbolic AutoAscend bot. By using mean scores per dataset as a normalization factor and rescaling to a range (similar to the D4RL methodology applied in the Katakomba benchmark), researchers can precisely quantify how PaCT closes the gap between neural models and hard-coded symbolic logic.⁴⁴ Previous interventions by Wolczyk et al. demonstrated that explicit knowledge retention methods could double the state-of-the-art score for neural models on the NetHack Human Monk scenario from 5,000 to over 10,500 points.³⁰ Incorporating the Share algorithm structurally automates this retention, theoretically pushing the boundaries of neural performance far closer to symbolic upper bounds without the need for exhaustive hyperparameter tuning of KL-divergence penalties.

Conclusion

The synthesis of the Universal Weight Subspace Hypothesis and the Parameter-Efficient Continual Finetuning (PaCT) paradigm via Shared LoRA Subspaces fundamentally reframes the theoretical understanding of deep neural network optimization and lifecycle management. The empirical confirmation that over 1,100 disparate neural models systematically collapse into identical, low-dimensional spectral manifolds dictates that high-dimensional parametric optimization is largely an illusion. Architecture and optimization algorithms, rather than domain data, govern the strict geometric bounds of what a network can represent.

The realization of the Share framework operationalizes this discovery. By analytically extracting the principal basis vectors of this universal subspace and dynamically expanding them only when mathematically essential, the framework achieves up to a 100x parameter reduction and a 281x memory savings. More imperatively, the use of Moore-Penrose pseudo-inverse reprojections provides an analytical, algebraic shield against catastrophic forgetting, allowing

single models to assimilate hundreds of sequential tasks without relying on massive data replay buffers.

When these subspace principles are mapped onto the volatile, non-stationary domain of reinforcement learning, they address the field's most critical bottlenecks. The Forgetting of Pre-trained Capabilities (FPC), specifically the state coverage gap and imperfect cloning gap, has historically crippled the fine-tuning of RL agents. By constraining Proximal Policy Optimization updates within a mathematically preserved low-rank shared basis, agents can achieve seamless forward and backward transfer in highly procedural, stochastic environments like the NetHack Learning Environment. The integration of Shared LoRA subspaces into complex recurrent and convolutional architectures, such as the Chaotic Dwarven GPT-5, theoretically guarantees that long-term temporal associations and structural heuristics are preserved regardless of adversarial task sampling.

Ultimately, the confluence of universal spectral convergence and analytical subspace reprojection paves the way for truly lifelong learning systems. As the computational bounds of artificial intelligence continue to expand, relying on localized, low-rank geometric invariants will not merely be an optimization technique, but an indispensable architectural necessity for maintaining continuous, reliable, and highly adaptive cognitive architectures.

Works cited

1. What is parameter-efficient fine-tuning (PEFT)? - IBM, accessed June 1, 2026, <https://www.ibm.com/think/topics/parameter-efficient-fine-tuning>
2. Catastrophic Forgetting in Kolmogorov-Arnold Networks, accessed June 1, 2026, <https://ojs.aaai.org/index.php/AAAI/article/view/39697/43658>
3. Shared LoRA Subspaces for almost Strict Continual Learning - arXiv, accessed June 1, 2026, <https://arxiv.org/html/2602.06043v1>
4. LoRA-Guided PPO for Cost-Aware and Compute-Efficient Agent Orchestration, accessed June 1, 2026, <https://neurips.cc/virtual/2025/126604>
5. RL(HF) algorithms with LoRA Support - verl documentation - Read the Docs, accessed June 1, 2026, https://verl.readthedocs.io/en/latest/advance/ppo_lora.html
6. RL Learning with LoRA: A Diverse Deep Dive, accessed June 1, 2026, <https://kalomaze.bearblog.dev/rl-lora-ddd/>
7. LoRA Without Regret - Thinking Machines Lab, accessed June 1, 2026, <https://thinkingmachines.ai/blog/lora/>
8. [2504.15777] Tina: Tiny Reasoning Models via LoRA - arXiv, accessed June 1, 2026, <https://arxiv.org/abs/2504.15777>
9. The Universal Weight Subspace Hypothesis - Prakhar Kaushik, accessed June 1, 2026, <https://toshi2k2.github.io/unisub/>
10. Universal Weight Subspace Hypothesis - Emergent Mind, accessed June 1, 2026, <https://www.emergentmind.com/topics/universal-weight-subspace-hypothesis>
11. The Universal Weight Subspace Hypothesis - arXiv, accessed June 1, 2026, <https://arxiv.org/pdf/2512.05117>
12. "The Universal Weight Subspace Hypothesis" : r/compsci - Reddit, accessed June

- 1, 2026,
https://www.reddit.com/r/compsci/comments/1pezmkp/the_universal_weight_subspace_hypothesis/
13. [2512.05117] The Universal Weight Subspace Hypothesis - arXiv, accessed June 1, 2026, <https://arxiv.org/abs/2512.05117>
 14. Neural Networks Keep Finding the Same Weight Geometry (No Matter What You Train Them On) : r/ArtificialSentience - Reddit, accessed June 1, 2026, https://www.reddit.com/r/ArtificialSentience/comments/1phwxkt/neural_networks_keep_finding_the_same_weight/
 15. Paper page - The Universal Weight Subspace Hypothesis - Hugging Face, accessed June 1, 2026, <https://huggingface.co/papers/2512.05117>
 16. The Universal Weight Subspace Hypothesis - arXiv, accessed June 1, 2026, <https://arxiv.org/html/2512.05117v1>
 17. The Universal Weight Subspace Hypothesis - GPTshop.ai, accessed June 1, 2026, <https://gptshop.ai/paper/Weight%20subspace%20hypothesis.pdf>
 18. The universal weight subspace hypothesis | Hacker News, accessed June 1, 2026, <https://news.ycombinator.com/item?id=46199623>
 19. Shared LoRA Subspaces for almost Strict Continual Learning - Emergent Mind, accessed June 1, 2026, <https://www.emergentmind.com/papers/2602.06043>
 20. [2602.06043] Shared LoRA Subspaces for almost Strict Continual Learning - arXiv, accessed June 1, 2026, <https://arxiv.org/abs/2602.06043>
 21. Shared LoRA Subspaces for almost Strict Continual Learning - Prakhar Kaushik, accessed June 1, 2026, <https://toshi2k2.github.io/share/>
 22. Daily Papers - Hugging Face, accessed June 1, 2026, <https://huggingface.co/papers?q=LoRA-based%20continual%20pre-training>
 23. Two Papers That Changed How I Think About Multi-Agent Coordination - Moltbook, accessed June 1, 2026, <https://www.moltbook.com/post/3794d418-7210-4edc-b492-de00583b795b>
 24. (PDF) Shared LoRA Subspaces for almost Strict Continual Learning - ResearchGate, accessed June 1, 2026, https://www.researchgate.net/publication/400505954_Shared_LoRA_Subspaces_for_almost_Strict_Continual_Learning
 25. The Universal Weight Subspace Hypothesis - arXiv, accessed June 1, 2026, <https://arxiv.org/html/2512.05117v2>
 26. Mining Attribute Subspaces for Efficient Fine-tuning of 3D Foundation Models - arXiv, accessed June 1, 2026, <https://arxiv.org/html/2604.10095v2>
 27. Prakhar Kaushik's research while affiliated with Johns Hopkins University and other places, accessed June 1, 2026, <https://www.researchgate.net/scientific-contributions/Prakhar-Kaushik-2174846049>
 28. The Official PyTorch implementation of Shared LoRA Subspaces for almost Strict Continual Learning - GitHub, accessed June 1, 2026, <https://github.com/ankit-vaitya19/Share>
 29. Scaling Laws for Forgetting When Fine-Tuning Large Language Models - arXiv, accessed June 1, 2026, <https://arxiv.org/html/2401.05605v1>

30. Fine-tuning Reinforcement Learning Models is Secretly a Forgetting Mitigation Problem, accessed June 1, 2026, <https://sites.google.com/view/finetune-rl-without-forgetting>
31. Fine-tuning Reinforcement Learning Models is Secretly a Forgetting, accessed June 1, 2026, <https://papers.lunadong.com/paper/15249>
32. Fine-tuning Reinforcement Learning Models is Secretly a Forgetting Mitigation Problem, accessed June 1, 2026, <https://arxiv.org/html/2402.02868v2>
33. [Literature Review] Fine-tuning Reinforcement Learning Models is Secretly a Forgetting Mitigation Problem - Moonlight, accessed June 1, 2026, <https://www.themoonlight.io/en/review/fine-tuning-reinforcement-learning-models-is-secretly-a-forgetting-mitigation-problem>
34. Mitigating Forgetting Between Supervised and Reinforcement Learning Yields Stronger Reasoners | OpenReview, accessed June 1, 2026, <https://openreview.net/forum?id=HHx5KFnj8P>
35. Maciej and Bartek - Fine-tuning Reinforcement Learning Models is a Forgetting Mitigation Problem - YouTube, accessed June 1, 2026, <https://www.youtube.com/watch?v=PzcJeRglfQI>
36. Revisiting The NetHack Learning Environment | ICLR Blogposts 2026, accessed June 1, 2026, <https://iclr-blogposts.github.io/2026/blog/2026/revisiting-the-nle/>
37. MiniHack the Planet: A Sandbox for Open-Ended Reinforcement Learning Research - GitHub, accessed June 1, 2026, <https://github.com/facebookresearch/minihack>
38. Generalization to New Sequential Decision Making Tasks with In-Context Learning - arXiv, accessed June 1, 2026, <https://arxiv.org/html/2312.03801v1>
39. MiniHack: A new sandbox for open-ended reinforcement learning - Meta AI, accessed June 1, 2026, <https://ai.meta.com/blog/minihack-a-new-sandbox-for-open-ended-reinforcement-learning/>
40. Online Intrinsic Rewards for Decision Making Agents from Large Language Model Feedback, accessed June 1, 2026, <https://arxiv.org/html/2410.23022v4>
41. Insights from the NeurIPS 2021 NetHack Challenge - Proceedings of Machine Learning Research, accessed June 1, 2026, <https://proceedings.mlr.press/v176/hambro22a/hambro22a.pdf>
42. Online Intrinsic Rewards for Decision Making Agents from Large Language Model Feedback - Reinforcement Learning Journal (RLJ), accessed June 1, 2026, https://rlj.cs.umass.edu/2025/papers/RLJ_RLC_2025_6.pdf
43. Katakomba: Tools and Benchmarks for Data-Driven NetHack - arXiv, accessed June 1, 2026, <https://arxiv.org/pdf/2306.08772>
44. NeurIPS Poster Katakomba: Tools and Benchmarks for Data-Driven NetHack, accessed June 1, 2026, <https://neurips.cc/virtual/2023/poster/73724>
45. Fine-tuning Reinforcement Learning Models is Secretly a Forgetting Mitigation Problem, accessed June 1, 2026, <https://proceedings.mlr.press/v235/wolczyk24a.html>